

Reviewer Pack — Minimal Tropical Hodge Structure Rank and DPLL Proof Path Le...

Entry 59b7022cf20c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 21:55:50 UTC

Minimal Tropical Hodge Structure Rank and DPLL Proof Path Length Inequality

Entry ID: 59b7022cf20c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 21:55:50 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every CNF φ with n variables, the minimal tropical Hodge structure rank ($h(\varphi)$) of its associated Tseitin formula is linearly correlated with its DPLL proof path length $l(\varphi)$, such that $h(\varphi) = \Omega(l(\varphi))$.

Rationale (proposer's reasoning):

Tropical Hodge structures provide a geometric interpretation of Boolean functions and their complexity. By exploring the relationship between these structures and the DPLL proof complexity, we may uncover new insights into the nature of unsatisfiability problems.

Taxonomy category: Tropical Geometry $\times$ Boolean Satisfiability (DPLL Proof Complexity)
(status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4ab360cf0985c1b7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all CNFs with n variables, the minimal tropical Hodge structure rank $h(\varphi)$ is greater than or equal to the DPLL proof path length $l(\varphi)$, and the correlation coefficient between them across 30 seeds is at least 0.8.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropical geometry' AND 'Boolean satisfiability' AND 'DPLL proof complexity' AND 'Hodge structure rank' - 'minimal tropical Hodge structure rank' AND DPLL AND 'proof path length' AND CNF-Tseitin formula AND 'linear correlation' AND 'DPLL proof' AND 'tropical geometry'

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1001.1554v4] Tropical geometry and correspondence theorems via toric stacks - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/2005.03737v2] A brief note on the second fundamental form of the variation of Hodge structure of a smooth projective hypersurface - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/2407.17951v1] Pruning Boolean d-DNNF Circuits Through Tseitin-Awareness - [http://arxiv.org/abs/2209.05839v3] On bounded depth proofs for Tseitin formulas on the grid; revisited - [http://arxiv.org/abs/math/0606330v4] The Baker-Campbell-Hausdorff formula in the free metabelian Lie algebra

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            # Find a non-zero pivot below
            for j in range(i + 1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                raise ValueError("Matrix is singular")
        factor = Fraction(1, A[i][i])
        for k in range(n):
            A[i][k] *= factor
        for j in range(n):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    return A
```

```

def fraction(a, b):
    gcd = math.gcd(a, b)
    return (a // gcd, b // gcd)

class Fraction:
    def __init__(self, numerator=0, denominator=1):
        if denominator == 0:
            raise ZeroDivisionError("Denominator cannot be zero")
        self.numerator, self.denominator = fraction(numerator, denominator)

    def __add__(self, other):
        if isinstance(other, int):
            other = Fraction(other)
        return Fraction(self.numerator * other.denominator + other.numerator * self.denominator,
                        self.denominator * other.denominator)

    def __mul__(self, other):
        if isinstance(other, int):
            other = Fraction(other)
        return Fraction(self.numerator * other.numerator, self.denominator * other.denominator)

    def __truediv__(self, other):
        if isinstance(other, int):
            other = Fraction(other)
        return Fraction(self.numerator * other.denominator, self.denominator * other.numerator)

    def __repr__(self):
        return f"Fraction({self.numerator}, {self.denominator})"

def tseitin_formula(phi):
    # Construct Tseitin formula from CNF phi
    pass

def tropical_hodge_structure_rank(formula):
    # Compute minimal tropical Hodge structure rank of the formula
    pass

def dpll_proof_path_length(phi):
    # Run DPLL algorithm on phi and return proof path length
    pass

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10 # Start with a small size and increase
    instances_tested = 0
    h_phi_sum = 0
    l_phi_sum = 0
    n_max = 0
    conjecture_holds = True
    counterexample = ""

    for _ in range(30):
        phi = generate_random_cnf(n)
        tseitin_formula_ = tseitin_formula(phi)
        h_phi = tropical_hodge_structure_rank(tseitin_formula_)

```

```

l_phi = dpll_proof_path_length(phi)

if h_phi is None or l_phi is None:
    conjecture_holds = False
    counterexample = "Tropical Hodge Structure Rank or Proof Path Length computation failed"
    break

h_phi_sum += h_phi
l_phi_sum += l_phi
instances_tested += 1
n_max = max(n_max, n)

if instances_tested < 30:
    conjecture_holds = False
    counterexample = "Insufficient instances tested"

mean_h_phi = h_phi_sum / instances_tested
mean_l_phi = l_phi_sum / instances_tested

return {
    "metric_name": "Tropical Hodge Structure Rank vs DPLL Proof Path Length",
    "metric_value": mean_h_phi,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

def generate_random_cnf(n):
    # Generate a random CNF with n variables
    pass

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_h_phi = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_h_phi} std=NA support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']}\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ff7d897.py", line 132, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ff7d897.py", line 110, in run_trial
    mean_h_phi = h_phi_sum / instances_tested
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the required correlation coefficient could not be calculated. | next: Re-run the test to ensure it completes successfully and produces the necessary data for calculating the correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12148
2	preregistration	ollama_remote	glm4:latest	0	0	9757
3	novelty	ollama_remote	glm4:latest	0	0	8728
4	novelty	ollama_remote	glm4:latest	0	0	10001
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	94054
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15692
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15530
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12952
9	judge	ollama_remote	glm4:latest	0	0	13737

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 192599 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/59b7022cf20c.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/59b7022cf20c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/59b7022cf20c.tar.gz` (if generated)